

Sintaxis, operadores y tipos de datos

# Índice

|                             |   |
|-----------------------------|---|
| Variables.....              | 3 |
| Palabras reservadas.....    | 3 |
| Entrada.....                | 4 |
| Tipos.....                  | 4 |
| Asignaciones múltiples..... | 5 |
| Comprobación de tipos.....  | 5 |
| Indicación de Tipos.....    | 6 |

# Variables

En este tema vamos a empezar a trabajar con variables, que son la forma que tenemos de almacenar datos en nuestro programa.

Podemos imaginar una variable como una caja a la que le damos un nombre y que es capaz de almacenar un valor, que puede cambiar. Las variables tienen también un tipo, que en Python viene definido por el valor que le damos.

Aunque no tenemos que decirlo explícitamente, en Python las variables tienen tipo.

En la siguiente línea estamos creando un variable y le damos el valor **Pepe**

```
persona = "Pepe"
print("Hola")
print(persona)
```

Llamamos asignación al hecho de dar a una variable un valor dado.

Diremos que hemos declarado la variable **persona** y le asignamos su valor.

Posteriormente, podemos cambiar su valor sin más que asignarle otro nuevo.

Vemos cómo distinguimos cuando queremos imprimir un texto literal, y el valor de la variable.

Para indicar el valor de las variables de tipo texto ponemos comillas simples ' o dobles "

Los nombres de las variables pueden estar formados por letras (mayúsculas y minúsculas, entre las que se distingue), números y el signo "\_" de guion bajo, pero no pueden contener espacios, ni empezar con números. Se recomienda que no se usen letras con acentos o ñ, si bien, todo esto depende más del editor que usemos, y cómo codifique el fichero.

Se recomienda usar nombres claros y descriptivos. Podemos usar guion bajo o separarlas por las letras mayúsculas/minúsculas. Por ejemplo:

```
nombrePersona = "Pepe"

ó
nombre_persona = "Pepe"
```

De la misma forma, aunque no es obligatorio se suele espaciar las asignaciones y las operaciones para hacerlos más legibles.

## Palabras reservadas

Existen ciertas palabras que tienen un uso concreto en Python y por tanto no podemos usarlas como nombres de variables. Puedes encontrar el listado aquí para la versión 3.13 de Python.

### Palabras reservadas

|        |          |         |          |        |
|--------|----------|---------|----------|--------|
| False  | await    | else    | import   | pass   |
| None   | break    | except  | in       | raise  |
| True   | class    | finally | is       | return |
| and    | continue | for     | lambda   | try    |
| as     | def      | from    | nonlocal | while  |
| assert | del      | global  | not      | with   |
| async  | elif     | if      | or       | yield  |

Podemos ver la lista de las palabras reservadas para nuestra versión de Python usando **help('keywords')**.

## Entrada

Ya que tenemos dónde guardar datos, vamos a ver cómo podemos pedirlos al usuario usando el teclado para ello.

La función que nos permite hacerlo se llama **input** (poca imaginación con el nombre...), y podemos indicar que queremos que se muestre un texto cuando se pide el dato.

```
nombreUsuario = input("¿Cómo te llamas? ")
print("Hola " + nombreUsuario)
```

Recuerda dejar un cierto espacio entre el final de la pregunta y nuestra respuesta. Podemos usar un espacio, o hacer que se salte a la línea siguiente con `\n`

La función **input** siempre nos va dar un valor de tipo **Cadena**, independientemente de lo que introduzca el usuario, por lo que deberemos convertirlos al tipo de datos que necesitemos.

```
nombreUsuario = input("¿Cómo te llamas?\n")
print("Hola " + nombreUsuario)
```

## Tipos

Cuando asignamos un valor a una variable, le estamos dando un tipo a la variable. Los tipos pueden ser numéricos, que podrán ser **enteros** o **decimales**, de tipo **cadena**, o **booleanos** (de valor lógico) que sólo pueden ser **True** o **False**.

En Python las variables tienen un tipo que se infiere a partir del contexto en la asignación, es decir, no es necesario explícitamente decírselo.

Para expresar **números decimales** usaremos el `.` como separador decimal:

```
# variable decimal o float
pi = 3.1416
```

También podemos usar **variables lógicas o booleanas** que podrán ser **True** o **False** (hay que escribirlas así):

```
# variable booleana o lógica
respuesta = True
```

Cuando asignamos un **texto** usamos las **comillas**, pero no para los números:

```
edad = 10
nombre = "Juan"
print(nombre)
print(" tiene ")
print(edad)
```

Dado que muchas veces necesitamos imprimir varias cosas de manera consecutiva, podemos separar los valores por comas. El ejemplo anterior quedaría así:

```
edad = 10
nombre = "Juan"
print(nombre, " tiene ", edad)
```

Conviene destacar que es diferente hacer:

```
a = 10

que:

a = "10"
```

Con el primero podremos hacer **operaciones aritméticas**:

```
a = 10
b = 15
c = a + b
print(c)
```

Obtenemos el esperado resultado de **25**.

Pero si usamos **cadenas**, el resultado puede no ser el esperado:

```
a = "10"
b = "15"
c = a + b
print(c)
```

Obtenemos **1015** que es el resultado de concatenar las dos cadenas

También podemos hacer todas estas operaciones en la consola de Python:

```
>>> a = 10
```

Como hemos visto, también existen las variables de tipo booleano, que sólo pueden contener 2 valores **True** o **False**, que han de ser escritas de esta manera:

```
isOpen = True

isVisible = False
```

## Asignaciones múltiples

Podemos asignar el valor a varias variables en una misma línea, sin más que separar los nombres a un lado del igual por comas, y también los valores al otro lado:

```
edad, nombre = 10, "Juan"
```

```
valor1 = valor2 = valor3 = 6
```

## Comprobación de tipos

Podemos comprobar el tipo de cualquier variable usando la función **type()**.

Así:

```
>>> text = 'hola'
>>> precio = 3.14
>>> posicion = 100
>>> print(type(precio))
<class 'float'>
>>> print(type(text))
<class 'str'>
>>> print(type(posicion))
<class 'int'>
```

Podemos comprobar si es de un tipo u otro usando la función **isinstance(variable, tipo)**, por ejemplo así:

```
>>> print(isinstance(posicion,int))
True
>>> print(isinstance(posicion,float))
False
>>> print(isinstance(precio,float))
True
>>> print(isinstance(text,str))
True
```

## Indicación de Tipos

A partir de la versión 3.0 de Python, podemos indicar el tipo que va a tener una variable. Sigue sin ser necesario hacerlo pero aporta ventajas como el evitar errores en nuestro código.

```
>>> text: str = 'hola'
>>> precio: float = 3.14
>>> posicion: int = 100
>>> condición: bool = True
```